

Федеральное государственное автономное образовательное учреждение высшего
образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий

Кафедра «Информатика и информационные технологии»

Направление подготовки/ специальность:

Автоматизированные системы обработки информации и управления

ОТЧЕТ

по проектной практике

Студенты: Савинова Вера Александровна, Барсегян Артур Самвелович,
Залипаев Алексей Сергеевич

Группа: 251-339

Место прохождения практики: Московский Политех, кафедра «Информатика и
информационные технологии»

Отчет принят с оценкой _____ Дата _____

Руководитель практики: _____

Москва 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ

1. Общая информация о проекте:
 - Название проекта
 - Цели и задачи проекта
2. Общая характеристика деятельности организации (*заказчика проекта*)
 - Наименование заказчика
 - Организационная структура
 - Описание деятельности
3. Описание задания по проектной практике
4. Описание достигнутых результатов по проектной практике

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Введение

Проектная практика проводилась в Московском Политехе на кафедре «Информатика и информационные технологии» в период с 16.02.2026 по 23.05.2026 2026 года. Трудоёмкость практики составила 72 академических часа. В ходе практики выполнялось два направления работы:

1. Участие в проекте Fungi (интеллектуальный онлайн-сервис по распознаванию грибов).
2. Реализация вариативной части — проекта «Оконный менеджер на C++ и SFML» в составе группы из трёх человек.

1. Общая информация о проекте

Название проекта:

Fungi — интеллектуальный онлайн-сервис-энциклопедия для распознавания грибов по фотографии с помощью ИИ.

Цель и задачи проекта:

Цель: создание доступного и удобного инструмента для идентификации грибов по фотографии.

Задачи:

- снижение риска ошибок при сборе грибов;
- популяризация знаний в области микологии;
- создание удобной базы данных грибов;
- внедрение технологий искусственного интеллекта в образовательные сервисы;
- развитие сообщества любителей природы и грибников.

2. Общая характеристика деятельности организации (заказчика проекта)

Наименование заказчика:

Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет».

Организационная структура:

Университет объединяет факультеты, институты, научные центры и лаборатории. Высшее руководство осуществляет ректорат, учебный процесс и проектную деятельность координируют профильные кафедры и проектные офисы.

Описание деятельности:

Московский Политех ведёт подготовку кадров по инженерным, IT- и технологическим направлениям, реализует образовательные программы, научные исследования и проектную деятельность с участием студентов. Вуз взаимодействует с индустриальными партнёрами, организует стажировки и практики, нацеленные на формирование практических навыков в области информационных технологий, программирования и цифровой трансформации.

3. Описание задания по проектной практике

Задание состояло из базовой части (работа над проектом Fungi) и вариативной части (проект «Оконный менеджер»).

Распределение ролей:

- **Документатор** - Савинова Вера Александровна
Ответственная за отчётность о прохождении учебной практики.
- **Full-Stack разработчик** - Залипаев Алексей Сергеевич
Ответственный за код и дизайн сайта, а так же за создание репозитория на Github.
- **C++ разработчик** - Барсегян Артур Самвелович
Ответственный за вариативное задание.

Ссылка на репозиторий в GitHub:

[GitHub - aqzsxcd/practice-Fungi · GitHub](#)

3.1 Базовая часть: участие в проекте Fungi

Формирование идеи

Создание концепции веб-приложения-энциклопедии грибов с системой распознавания изображений.

Проектирование интерфейса

Разработка макетов страниц, структуры меню, навигации. Уделялось внимание простоте, удобству и единому стилю.

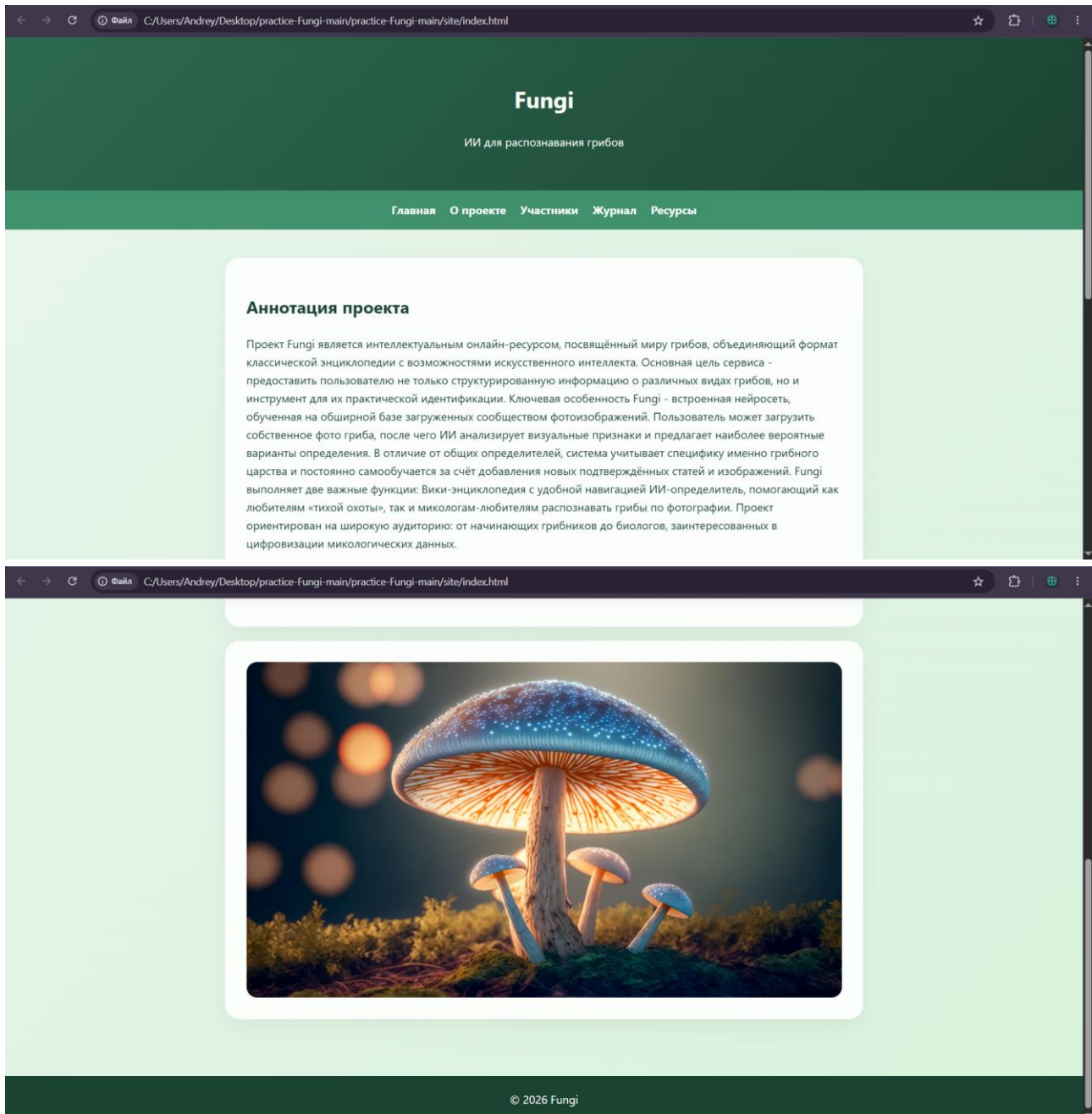
Разработка сайта

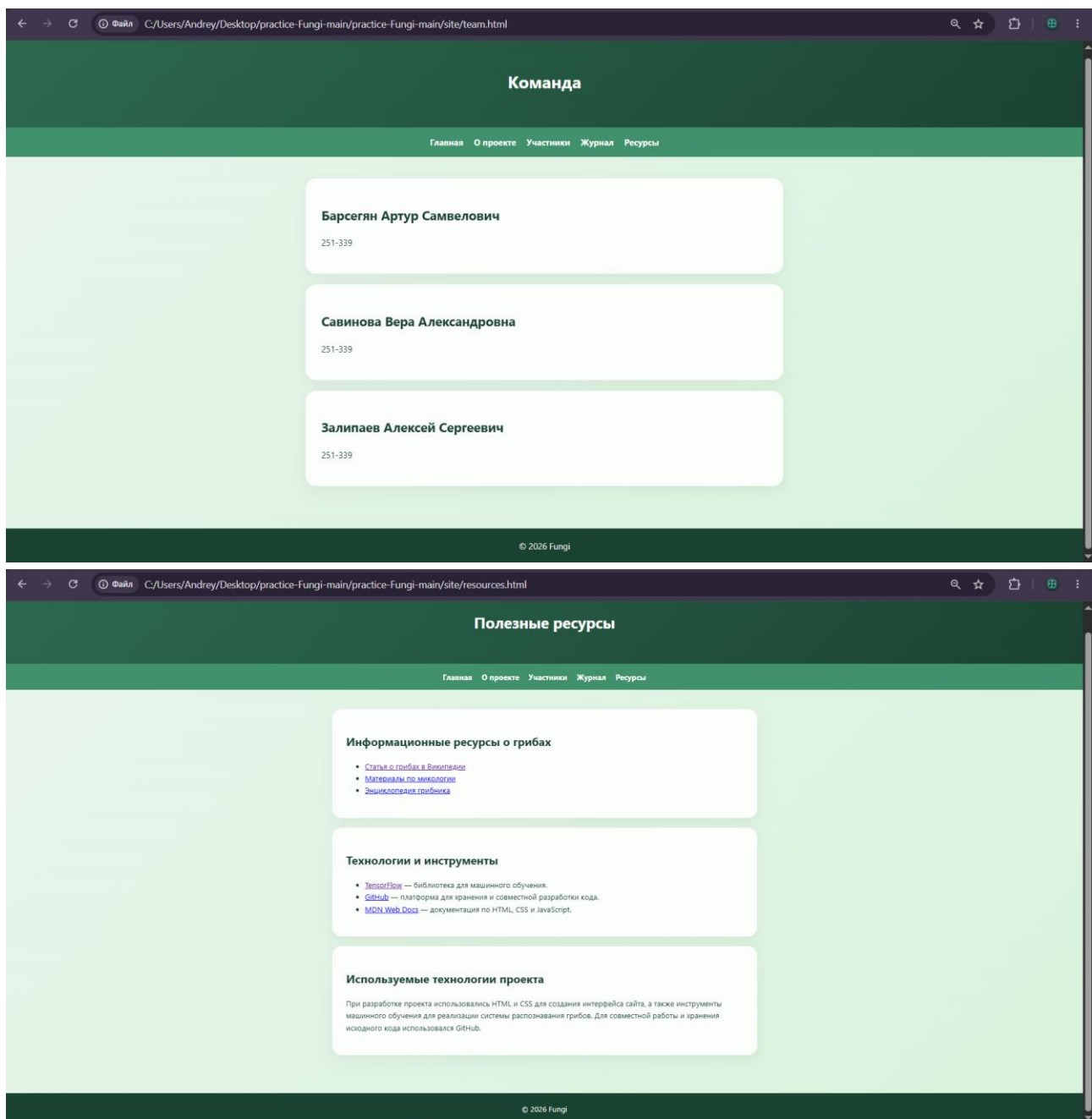
Создание страниц на HTML и CSS.

Настройка навигации.

Подключение контента.

Скриншоты сайта:





Работа с GitHub

Освоение команд `git clone`, `add`, `commit`, `push`.
Работа с ветками и слияние изменений.

Разработка ИИ-модуля

Изучение TensorFlow, Keras.
Сбор изображений грибов.
Обучение нейросети для распознавания.

Тестирование

Проверка отображения страниц, навигации, исправление ошибок.

Хронология по журналу проекта:

Неделя Содержание

1	Формирование идеи, анализ аналогов
2	Исследование грибов, сбор изображений, изучение нейросетей
3	Разработка структуры сайта (HTML/CSS)
4	Обучение модели ИИ, столкнулись с нехваткой данных
5	Тестирование, улучшение оформления и навигации
6	Активная работа с GitHub, документация в Markdown
7	Финальное оформление, подготовка к публикации

3.2 Вариативная часть: проект «Оконный менеджер»

Состав группы:

Залипаев Алексей Сергеевич — разработчик

Барсегян Артур Самвелович — тестировщик

Савинова Вера Александровна — документатор

Цель: создание приложения для управления окнами с возможностью создавать, закрывать, перетаскивать окна и оригинальной анимацией «встряски».

Этапы работ и временные затраты:

№	Этап	Часы
1	Установка и настройка среды разработки (VS2022, SFML)	2
2	Изучение документации SFML	3
3	Проектирование архитектуры (черновые	2

№	Этап	Часы
	наброски, UML)	
4	Создание базового каркаса приложения	3
5	Реализация создания окон	3
6	Добавление заголовков и кнопки закрытия	3
7	Реализация закрытия окон	2
8	Реализация перетаскивания окон	4
9	Модификация (творческая часть) – анимация встряски	4
10	Расширенное тестирование и отладка	3
11	Создание UML-диаграмм (4 типа)	3
12	Написание технического руководства (черновик)	2
13	Написание технического руководства (чистовой вариант)	2
14	Создание HTML-версии документации	2
15	Подготовка видео-презентации	2
16	Финальная проверка и сдача	2
Итого		40

Реализованная функциональность:

Создание окон по клавише N.

Закрытие окон через кнопку [X].

Перетаскивание за заголовок.

Поднятие активного окна наверх при клике.

Встряска окна при клике по телу окна (сдвиг на ± 8 пикселей, 12 кадров).

Случайный цвет для каждого окна.

Проблемы и решения:

Ошибка LNK2005 (main already defined) — исключили Default.cpp из сборки.

Перетаскивание ломалось после модификации — сброс флагов перед каждым кликом.

Встряска срабатывала на верхнем окне, а не на кликнутом — применена двухпроходная проверка.

Листинг кода проекта:

```
#include <SFML/Graphics.hpp> // подключение графической библиотеки SFML
#include <vector> // подключение библиотеки vector
#include <cstdlib> // генерация случайных чисел/цветов
#include <ctime> // инициализации генератора случайных чисел
#include <string>
#include <iostream>
```

```
struct Okno
{
    int x, y; // координаты левого верхнего угла
    int w, h; // ширина и высота окна
    bool aktivno; // активно ли окно
    bool zakryto; // помечено ли окно на закрытие
    std::string nazvanie; // имя окна
    bool peretaskivaetsya; // перетаскивается ли окно сейчас
    int otnosX, otnosY; // смещение клика относительно угла окна
    int vstryaskaTimer = 0; // таймер для анимации встряски
};
```

```
int main()
{
    // Переменные для перетаскивания окон
    int indeksPeretaskivaemogoOkna = -1; // -1 - окно не перетаскивается

    // Главное окно приложения
    sf::RenderWindow desktop(sf::VideoMode(1024, 768), "Oconnyj Menedzher");

    // Список всех управляемых окон
    std::vector<Okno> okna;

    // Цвета для каждого окна
    std::vector<sf::Color> cveta;

    // Инициализация генератора случайных чисел для цветов окон
```

```

std::srand(static_cast<unsigned>(std::time(nullptr)));

// Загрузка шрифта для текста в заголовках окон
sf::Font shrift;
if (!shrift.loadFromFile("arial.ttf")) // пытаемся загрузить стандартный шрифт
{
}

// Главный цикл приложения
while (desktop.isOpen()) // открыто окно
{
    sf::Event sobytie; // хранение события

    // pollEvent - проверка и обработка событий, поступающих к окну приложения
    while (desktop.pollEvent(sobytie))
    {
        // закрытие окна
        if (sobytie.type == sf::Event::Closed)
        {
            desktop.close();
        }

        if (sobytie.type == sf::Event::KeyPressed)
        {
            // проверка на нажатие клавиши N
            if (sobytie.key.code == sf::Keyboard::N)
            {
                // Создание нового окна
                Okno novoeOkno;
                novoeOkno.x = 50 + okna.size() * 30; // сдвиг по X
                novoeOkno.y = 50 + okna.size() * 30; // сдвиг по Y
                novoeOkno.w = 200; // ширина
                novoeOkno.h = 150; // высота
                novoeOkno.aktivno = false; // пока неактивно
                novoeOkno.zakryto = false; // не закрыто
                novoeOkno.nazvanie = "Okno " + std::to_string(okna.size() + 1); // название
                novoeOkno.peretaskivaetsya = false; // изначально не перетаскивается
                novoeOkno.otnosX = 0; // начальное смещение по X
                novoeOkno.otnosY = 0; // начальное смещение по Y
                novoeOkno.vstryaskaTimer = 0;

                // Добавляем окно в список
                okna.push_back(novoeOkno);
            }
        }
    }
}

```

```

// создание случайного цвета
sf::Color sluchajnyjCvet(
    std::rand() % 200 + 55, // красный
    std::rand() % 200 + 55, // зелёный
    std::rand() % 200 + 55 // синий
);

cveta.push_back(sluchajnyjCvet);
}
}

// MouseButtonPressed - обрабатывает нажатие кнопки мыши
if (sobytie.type == sf::Event::MouseButtonPressed)
{
    // Сброс нажатия перед новым кликом
    indeksPeretaskivaemogoOkna = -1;
    for (size_t j = 0; j < okna.size(); ++j)
    {
        okna[j].peretaskivaetsya = false;
    }

    if (sobytie.mouseButton.button == sf::Mouse::Left)
    {
        int myshX = sobytie.mouseButton.x;
        int myshY = sobytie.mouseButton.y;

        // Поиск окна по какому кликнули для выдачи встряски
        int indexKliknutogoOkna = -1;
        for (int i = static_cast<int>(okna.size()) - 1; i >= 0; --i)
        {
            if (okna[i].zakryto) continue;

            int vysotaZagolovka = 25;

            // Проверяем, попадает ли клик в пределы окна (включая заголовок и тело)
            if (myshX >= okna[i].x && myshX <= okna[i].x + okna[i].w && myshY >=
okna[i].y && myshY <= okna[i].y + okna[i].h)
            {
                indexKliknutogoOkna = i;
                break;
            }
        }
    }
}

```

```

// Обработка действий с самого верхнего окна
for (int i = static_cast<int>(okna.size()) - 1; i >= 0; --i)
{
    if (okna[i].zakryto) continue;

    int vysotaZagolovka = 25;
    int knopkaW = 20;
    int knopkaH = 18;
    int knopkaX = okna[i].x + okna[i].w - knopkaW - 5;
    int knopkaY = okna[i].y + 3;

    // Проверка на кнопку закрытия
    if (myshX >= knopkaX && myshX <= knopkaX + knopkaW &&
        myshY >= knopkaY && myshY <= knopkaY + knopkaH)
    {
        okna[i].zakryto = true;
        break;
    }

    // Проверка заголовка на перетаскивание
    if (myshX >= okna[i].x && myshX <= okna[i].x + okna[i].w &&
        myshY >= okna[i].y && myshY <= okna[i].y + vysotaZagolovka)
    {
        // Поднимаем окно наверх
        Okno vremennoe = okna[i];
        okna.erase(okna.begin() + i);
        okna.push_back(vremennoe);

        // Обновка цветов
        sf::Color vremennyjCvet = cveta[i];
        cveta.erase(cveta.begin() + i);
        cveta.push_back(vremennyjCvet);

        // Новый индекс окна (последний в векторе)
        int novyjIndeks = static_cast<int>(okna.size()) - 1;

        // Активирование перетаскивания
        indeksPeretaskivaemogoOkna = novyjIndeks;
        okna[novyjIndeks].peretaskivaetsya = true;
        okna[novyjIndeks].otnosX = myshX - okna[novyjIndeks].x;
        okna[novyjIndeks].otnosY = myshY - okna[novyjIndeks].y;

        break;
    }
}

```

```

    }

    // Встряска: применение к окну, по которому кликнули
    if (indexKliknutogoOkna != -1 && !okna[indexKliknutogoOkna].zakryto)
    {
        int vysotaZagolovka = 25;
        // Проверяем, что клик был именно по телу (не по заголовку)
        if (myshY >= okna[indexKliknutogoOkna].y + vysotaZagolovka &&
            myshY <= okna[indexKliknutogoOkna].y + okna[indexKliknutogoOkna].h)
        {
            okna[indexKliknutogoOkna].vstryaskaTimer = 12;
        }
    }
}

// MouseButtonReleased - обрабатывает отпускание кнопки мыши
if (sobytie.type == sf::Event::MouseButtonReleased)
{
    if (sobytie.mouseButton.button == sf::Mouse::Left)
    {
        indeksPeretaskivaemogoOkna = -1;
        for (size_t j = 0; j < okna.size(); ++j)
        {
            okna[j].peretaskivaetsya = false;
        }
    }
}

// MouseMoved - обрабатывает движение мыши
if (sobytie.type == sf::Event::MouseMoved)
{
    if (indeksPeretaskivaemogoOkna != -1 &&
        indeksPeretaskivaemogoOkna < static_cast<int>(okna.size()))
    {
        if (!okna[indeksPeretaskivaemogoOkna].zakryto)
        {
            int myshX = sobytie.mouseMove.x;
            int myshY = sobytie.mouseMove.y;

            okna[indeksPeretaskivaemogoOkna].x = myshX -
okna[indeksPeretaskivaemogoOkna].otnosX;
            okna[indeksPeretaskivaemogoOkna].y = myshY -
okna[indeksPeretaskivaemogoOkna].otnosY;

```

```

    }
    else
    {
        indeksPeretaskivаемогоOkna = -1;
    }
}
}

// Удаление закрытых окон из списков
for (int i = static_cast<int>(okna.size()) - 1; i >= 0; --i)
{
    if (okna[i].zakryto)
    {
        okna.erase(okna.begin() + i);
        cveta.erase(cveta.begin() + i);
    }
}

}

desktop.clear(sf::Color(50, 50, 80)); // тёмно-синий фон

// Отрисовка всех окон
for (size_t i = 0; i < okna.size(); ++i) // size_t - размер вектора
{
    // Если окно не закрыто
    if (!okna[i].zakryto)
    {
        // ВСТРЯСКА: вычисляем сдвиг по X
        int sdvigX = 0;
        if (okna[i].vstryaskaTimer > 0)
        {
            sdvigX = (okna[i].vstryaskaTimer % 2 == 0) ? 8 : -8; // сдвиг на 8 пикселей
            okna[i].vstryaskaTimer -= 1;
        }

        // Координаты окна с учётом сдвига
        int tekX = okna[i].x + sdvigX;
        int tekY = okna[i].y;

        // Основной прямоугольник окна

```

```

sf::RectangleShape prjamougolnik(sf::Vector2f(okna[i].w, okna[i].h));
prjamougolnik.setPosition(tekX, tekY);
prjamougolnik.setFillColor(cveta[i]);
prjamougolnik.setOutlineThickness(2);
prjamougolnik.setOutlineColor(sf::Color::White);
desktop.draw(prjamougolnik);

int vysotaZagolovka = 25;

// Заголовок
sf::RectangleShape zagolovok(sf::Vector2f(okna[i].w, vysotaZagolovka));
zagolovok.setPosition(tekX, tekY);
zagolovok.setFillColor(sf::Color(80, 80, 120));
desktop.draw(zagolovok);

// Текст заголовка
sf::Text tekst;
tekst.setFont(shrift);
tekst.setString(okna[i].nazvanie);
tekst.setCharacterSize(14);
tekst.setFillColor(sf::Color::White);
tekst.setPosition(tekX + 5, tekY + 4);
desktop.draw(tekst);

// Кнопка закрытия
int knopkaW = 20;
int knopkaH = 18;
sf::RectangleShape knopka(sf::Vector2f(knopkaW, knopkaH));
knopka.setPosition(tekX + okna[i].w - knopkaW - 5, tekY + 3);
knopka.setFillColor(sf::Color(180, 60, 60));
knopka.setOutlineThickness(1);
knopka.setOutlineColor(sf::Color::White);
desktop.draw(knopka);

// Текст [X] на кнопке
sf::Text krestik;
krestik.setFont(shrift);
krestik.setString("X");
krestik.setCharacterSize(14);
krestik.setFillColor(sf::Color::White);
krestik.setPosition(tekX + okna[i].w - knopkaW - 5 + 6, tekY + 4);
desktop.draw(krestik);
}
}

```

```
// отображение на экране
desktop.display();
}

return 0;
}
```

4. Описание достигнутых результатов по проектной практике

По проекту Fungi:

Разработан статический веб-сайт на HTML/CSS с главной страницей, описанием проекта, журналом, разделом участников и ресурсов. Отработаны навыки командной разработки и ведения Git-репозитория. Сформирован контент в формате Markdown в папке docs/.

По вариативной части «Оконный менеджер»:

Написан рабочий оконный менеджер на C++ (SFML) объёмом более 300 строк. Все заявленные функции (создание, закрытие, перетаскивание, поднятие вверх, встряска) работают корректно. Подготовлено техническое руководство для начинающих (Markdown + HTML) с пошаговыми инструкциями и кодом. Разработаны UML-диаграммы (классов, последовательности, состояний, блок-схема). Записана видео-презентация, демонстрирующая работу приложения. Создана структура проекта, готовая к компиляции в Visual Studio 2022.

Оценка ценности для заказчика:

Проект Fungi может быть использован в учебных целях как пример интеграции ИИ в веб-приложение. Оконный менеджер является наглядной демонстрацией работы с графикой, событиями и анимацией на C++.

Заключение

За время проектной практики полностью выполнены задачи базовой и вариативной частей. Освоены: веб-разработка (HTML/CSS), основы ИИ (TensorFlow), Git, документирование в Markdown, программирование на C++ с библиотекой SFML. Созданные проекты обладают практической ценностью для образовательного процесса кафедры. Все результаты загружены в репозиторий на GitHub, оформлены в соответствии с требованиями задания.

Список использованной литературы:

Введение в CSS верстку:

https://developer.mozilla.org/ru/docs/Learn_web_development/Core/CSS_layout/Introduction

DevTools: <https://habr.com/ru/articles/548898/>

Элементы HTML: <https://developer.mozilla.org/ru/docs/Web/HTML/Element>

Основы HTML:

https://developer.mozilla.org/ru/docs/Learn_web_development/Getting_started/Your_first_website/Creating_the_content

Основы CSS: <https://developer.mozilla.org/ru/docs/Web/CSS> <https://doka.guide/>

Официальная документация Git: <https://git-scm.com/book/ru/v2> ;

https://skillbox.ru/media/code/chto_takoe_git_obyasnyаем_na_skhemakh/

Курс на Hexlet по Git: https://ru.hexlet.io/courses/intro_to_git

Документация Markdown: https://ru.hexlet.io/lesson_filters/markdown